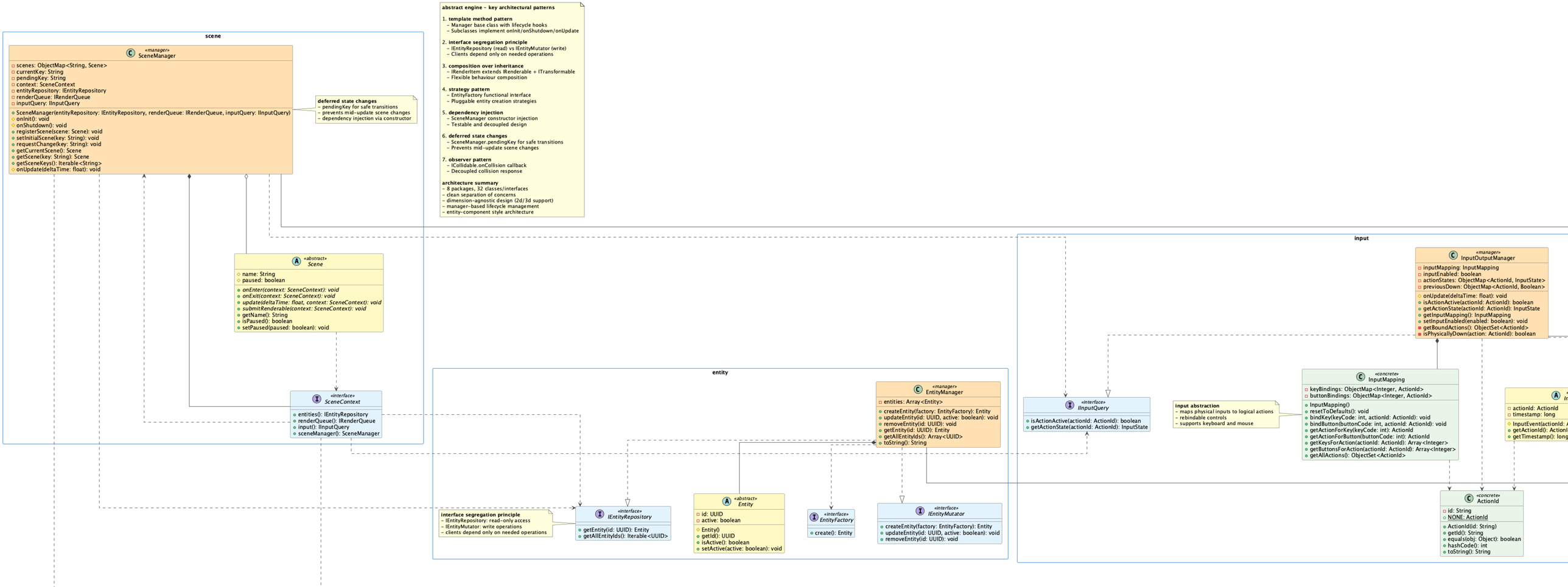
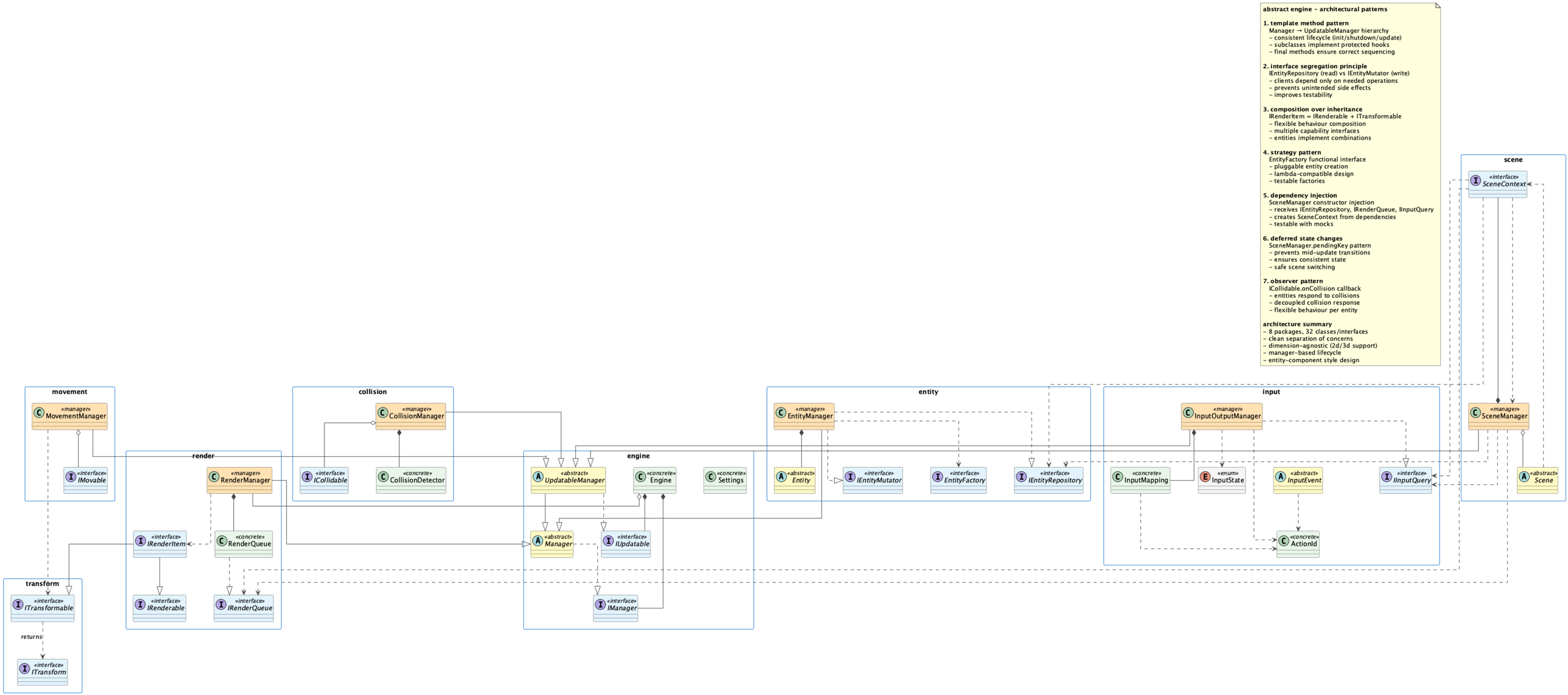

abstract engine - key architectural patterns
1. template method pattern
- Manager base class with lifecycle hooks
- Subclasses implement onInit/onShutdown/onUpdate
2. interface segregation principle
- IEntityRepository (read) vs IEntityMutator (write)
- Clients depend only on needed operations
3. composition over inheritance
- IRenderItem extends IRenderable + ITransformable
- Flexible behaviour composition
4. strategy pattern
- EntityFactory functional interface
- Pluggable entity creation strategies
5. dependency injection
- SceneManager constructor injection
- Testable and decoupled design
6. deferred state changes
- SceneManager.pendingKey for safe transitions
- Prevents mid-update scene changes
7. observer pattern
- ICollidable.onCollision callback
- Decoupled collision response
architecture summary
- 8 packages, 32 classes/interfaces
- clean separation of concerns
- dimension-agnostic design (2d/3d support)
- manager-based lifecycle management
- entity-component style architecture
scene
«manager» SceneManager
scenes: ObjectMap<String, Scene>
currentKey: String
pendingKey: String
context: SceneContext
entityRepository: IEntityRepository
renderQueue: IRenderQueue
inputQuery: IInputQuery
SceneManager(entityRepository: IEntityRepository, renderQueue: IRenderQueue, inputQuery: IInputQuery)
onInit(): void
onShutdown(): void
registerScene(scene: Scene): void
setInitialScene(key: String): void
requestChange(key: String): void
getCurrentScene(): Scene
getScene(key: String): Scene
getSceneKeys(): Iterable<String>
onUpdate(deltaTime: float): void
deferred state changes
- pendingKey for safe transitions
- prevents mid-update scene changes
- dependency injection via constructor
«abstract» Scene
name: String
paused: boolean
onEnter(context: SceneContext): void
onExit(context: SceneContext): void
update(deltaTime: float, context: SceneContext): void
submitRenderable(context: SceneContext): void
getName(): String
isPaused(): boolean
setPaused(paused: boolean): void
«interface» SceneContext
entities(): IEntityRepository
renderQueue(): IRenderQueue
input(): IInputQuery
sceneManager(): SceneManager
entity
«manager» EntityManager
entities: Array<Entity>
createEntity(factory: EntityFactory): Entity
updateEntity(id: UUID, active: boolean): void
removeEntity(id: UUID): void
getEntity(id: UUID): Entity
getAllEntityIds(): Array<UUID>
toString(): String
interface segregation principle
- IEntityRepository: read-only access
- IEntityMutator: write operations
- clients depend only on needed operations
«interface» IEntityRepository
getEntity(id: UUID): Entity
getAllEntityIds(): Iterable<UUID>
«abstract» Entity
id: UUID
active: boolean
Entity()
getId(): UUID
isActive(): boolean
setActive(active: boolean): void
«interface» EntityFactory
create(): Entity
«interface» IEntityMutator
createEntity(factory: EntityFactory): Entity
updateEntity(id: UUID, active: boolean): void
removeEntity(id: UUID): void
input
«manager» InputOutputManager
inputMapping: InputMapping
inputEnabled: boolean
actionStates: ObjectMap<ActionId, InputState>
previousDown: ObjectMap<ActionId, Boolean>
onUpdate(deltaTime: float): void
isActionActive(actionId: ActionId): boolean
getActionState(actionId: ActionId): InputState
getInputMapping(): InputMapping
setInputEnabled(enabled: boolean): void
getBoundActions(): ObjectSet<ActionId>
isPhysicallyDown(action: ActionId): boolean
«interface» IInputQuery
isActionActive(actionId: ActionId): boolean
getActionState(actionId: ActionId): InputState
input abstraction
- maps physical inputs to logical actions
- rebindable controls
- supports keyboard and mouse
«concrete» InputMapping
keyBindings: ObjectMap<Integer, ActionId>
buttonBindings: ObjectMap<Integer, ActionId>
InputMapping()
resetToDefaults(): void
bindKey(keyCode: int, actionId: ActionId): void
bindButton(buttonCode: int, actionId: ActionId): void
getActionForKey(keyCode: int): ActionId
getActionForButton(buttonCode: int): ActionId
getKeysForAction(actionId: ActionId): Array<Integer>
getButtonsForAction(actionId: ActionId): Array<Integer>
getAllActions(): ObjectSet<ActionId>
actionId: ActionId
timestamp: long
InputEvent(actionId: A
getActionId(): Action
getTimestamp(): long
«concrete» ActionId
id: String
NONE: ActionId
ActionId(id: String)
getId(): String
equals(obj: Object): boolean
hashCode(): int
toString(): String


abstract engine - architectural patterns
1. template method pattern
Manager → UpdatableManager hierarchy
- consistent lifecycle (init/shutdown/update)
- subclasses implement protected hooks
- final methods ensure correct sequencing
2. interface segregation principle
IEntityRepository (read) vs IEntityMutator (write)
- clients depend only on needed operations
- prevents unintended side effects
- improves testability
3. composition over inheritance
IRenderItem = IRenderable + ITransformable
- flexible behaviour composition
- multiple capability interfaces
- entities implement combinations
4. strategy pattern
EntityFactory functional interface
- pluggable entity creation
- lambda-compatible design
- testable factories
5. dependency injection
SceneManager constructor injection
- receives IEntityRepository, IRenderQueue, IInputQuery
- creates SceneContext from dependencies
- testable with mocks
6. deferred state changes
SceneManager.pendingKey pattern
- prevents mid-update transitions
- ensures consistent state
- safe scene switching
7. observer pattern
ICollidable.onCollision callback
- entities respond to collisions
- decoupled collision response
- flexible behaviour per entity
architecture summary
- 8 packages, 32 classes/interfaces
- clean separation of concerns
- dimension-agnostic (2d/3d support)
- manager-based lifecycle
- entity-component style design
movement
«manager» MovementManager
«interface» IMovable
collision
«manager» CollisionManager
«interface» ICollidable
«concrete» CollisionDetector
render
«manager» RenderManager
«interface» IRenderItem
«concrete» RenderQueue
«interface» IRenderable
«interface» IRenderQueue
transform
«interface» ITransformable
returns
«interface» ITransform
engine
«abstract» UpdatableManager
«concrete» Engine
«concrete» Settings
«abstract» Manager
«interface» IUpdatable
«interface» IManager
entity
«manager» EntityManager
«abstract» Entity
«interface» IEntityMutator
«interface» EntityFactory
«interface» IEntityRepository
input
«manager» InputOutputManager
«concrete» InputMapping
«enum» InputState
«abstract» InputEvent
«interface» IInputQuery
«concrete» ActionId
scene
«interface» SceneContext
«manager» SceneManager
«abstract» Scene

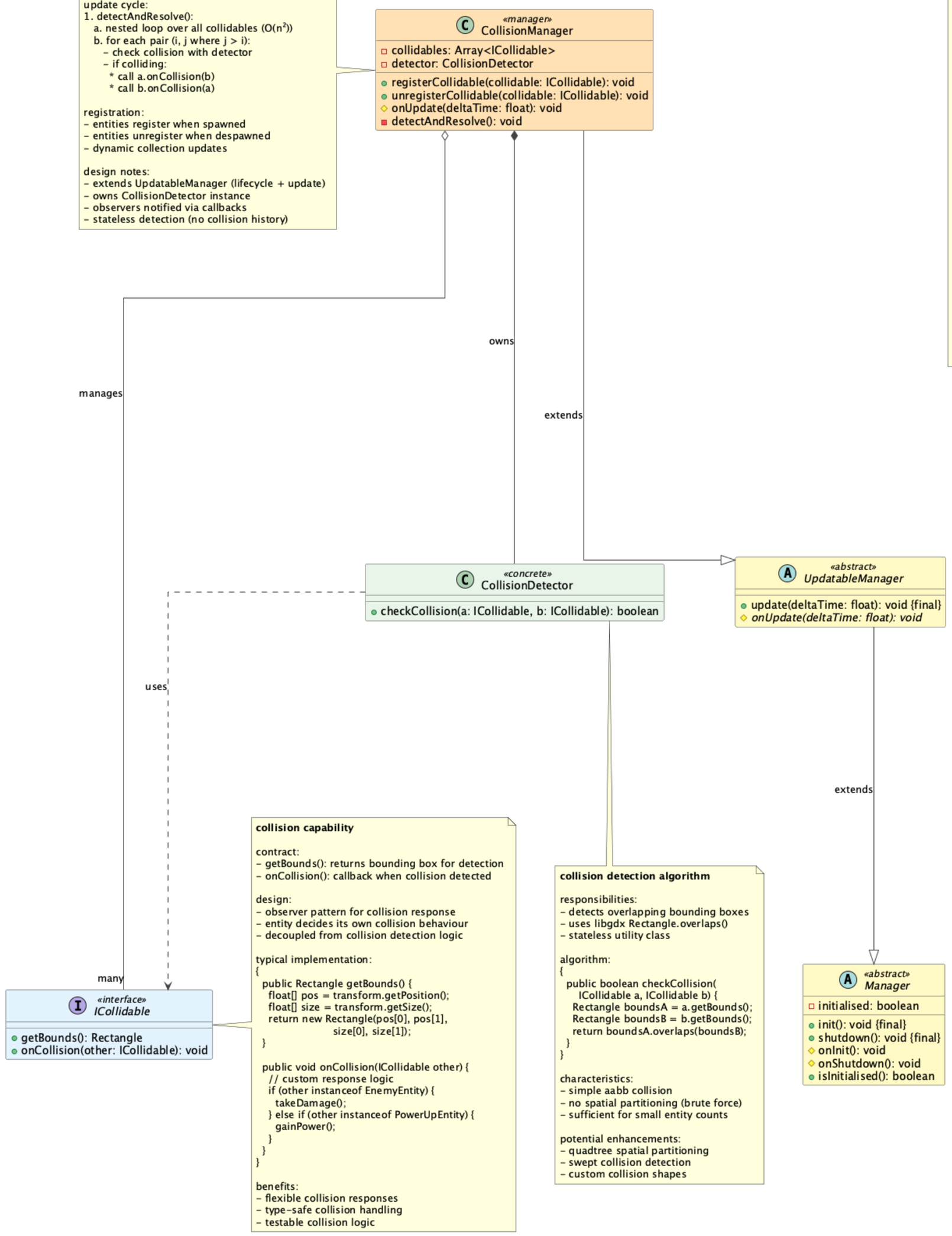
collision orchestrator

responsibilities:
- manages collection of collidable entities
- detects collisions each frame
- triggers collision callbacks

update cycle:
1. detectAndResolve():
   a. nested loop over all collidables ($O(n^2)$)
   b. for each pair (i, j where j > i):
      - check collision with detector
      - if colliding:
        * call a.onCollision(b)
        * call b.onCollision(a)

registration:
- entities register when spawned
- entities unregister when despawned
- dynamic collection updates

design notes:
- extends UpdatableManager (lifecycle + update)
- owns CollisionDetector instance
- observers notified via callbacks
- stateless detection (no collision history)



usage example

```
{
  // create collision manager
  CollisionManager collisionManager = new CollisionManager();
  collisionManager.init();

  // create collidable entity
  class Player extends Entity implements ICollidable {
    private Transform2D transform;

    public Rectangle getBounds() {
      float[] pos = transform.getPosition();
      float[] size = transform.getSize();
      return new Rectangle(pos[0], pos[1],
                           size[0], size[1]);
    }

    public void onCollision(ICollidable other) {
      if (other instanceof Enemy) {
        System.out.println("hit by enemy!");
        health -= 10;
      } else if (other instanceof Coin) {
        System.out.println("collected coin!");
        score += 100;
      }
    }
  }

  // register entities
  Player player = new Player();
  Enemy enemy = new Enemy();
  Coin coin = new Coin();

  collisionManager.registerCollidable(player);
  collisionManager.registerCollidable(enemy);
  collisionManager.registerCollidable(coin);

  // game loop
  while (running) {
    float deltaTime = getDeltaTime();
    collisionManager.update(deltaTime);
    // collisions detected and callbacks invoked
  }

  // cleanup
  collisionManager.unregisterCollidable(player);
  collisionManager.shutdown();
}
```

collision detection algorithm

```
{
  private void detectAndResolve() {
    int size = collidables.size;

    // nested loop: check all pairs
    for (int i = 0; i < size - 1; i++) {
      ICollidable a = collidables.get(i);

      for (int j = i + 1; j < size; j++) {
        ICollidable b = collidables.get(j);

        // detect collision
        if (detector.checkCollision(a, b)) {
          // notify both entities
          a.onCollision(b);
          b.onCollision(a);
        }
      }
    }
  }
}
```

complexity:
- time: $O(n^2)$ where n = entity count
- space: O(1) no additional storage

optimisation opportunities:
- spatial partitioning (quadtree, grid)
- broad phase / narrow phase
- sleeping entities (static objects)
- collision layers/masks

design patterns in collision system

1. **observer pattern**
   - ICollidable.onCollision() as observer
   - CollisionManager as subject
   - entities notified of collisions

2. **strategy pattern**
   - onCollision() as strategy
   - each entity implements custom response
   - flexible collision behaviours

3. **template method pattern**
   - UpdatableManager.update() calls onUpdate()
   - onUpdate() calls detectAndResolve()
   - consistent update lifecycle

4. **separation of concerns**
   - CollisionDetector: detection logic
   - CollisionManager: orchestration
   - ICollidable: response logic

benefits:
- decoupled collision logic
- testable components
- flexible entity responses
- simple aabb detection
- extensible detection algorithms

collision capability

contract:
- getBounds(): returns bounding box for detection
- onCollision(): callback when collision detected

design:
- observer pattern for collision response
- entity decides its own collision behaviour
- decoupled from collision detection logic

typical implementation:

```
{
  public Rectangle getBounds() {
    float[] pos = transform.getPosition();
    float[] size = transform.getSize();
    return new Rectangle(pos[0], pos[1],
                         size[0], size[1]);
  }

  public void onCollision(ICollidable other) {
    // custom response logic
    if (other instanceof EnemyEntity) {
      takeDamage();
    } else if (other instanceof PowerUpEntity) {
      gainPower();
    }
  }
}
```

benefits:
- flexible collision responses
- type-safe collision handling
- testable collision logic

collision detection algorithm

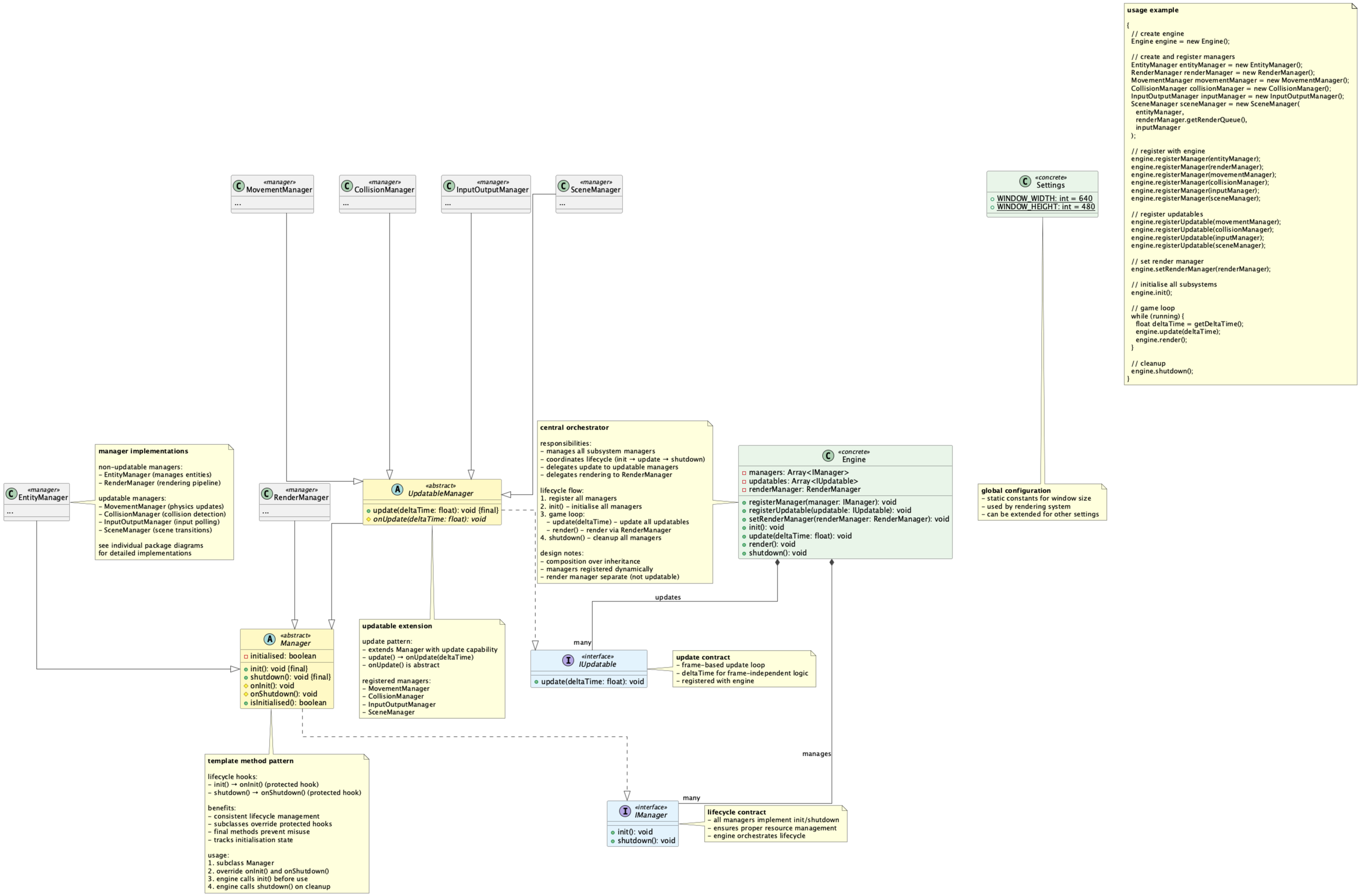
responsibilities:
- detects overlapping bounding boxes
- uses libgdx Rectangle.overlaps()
- stateless utility class

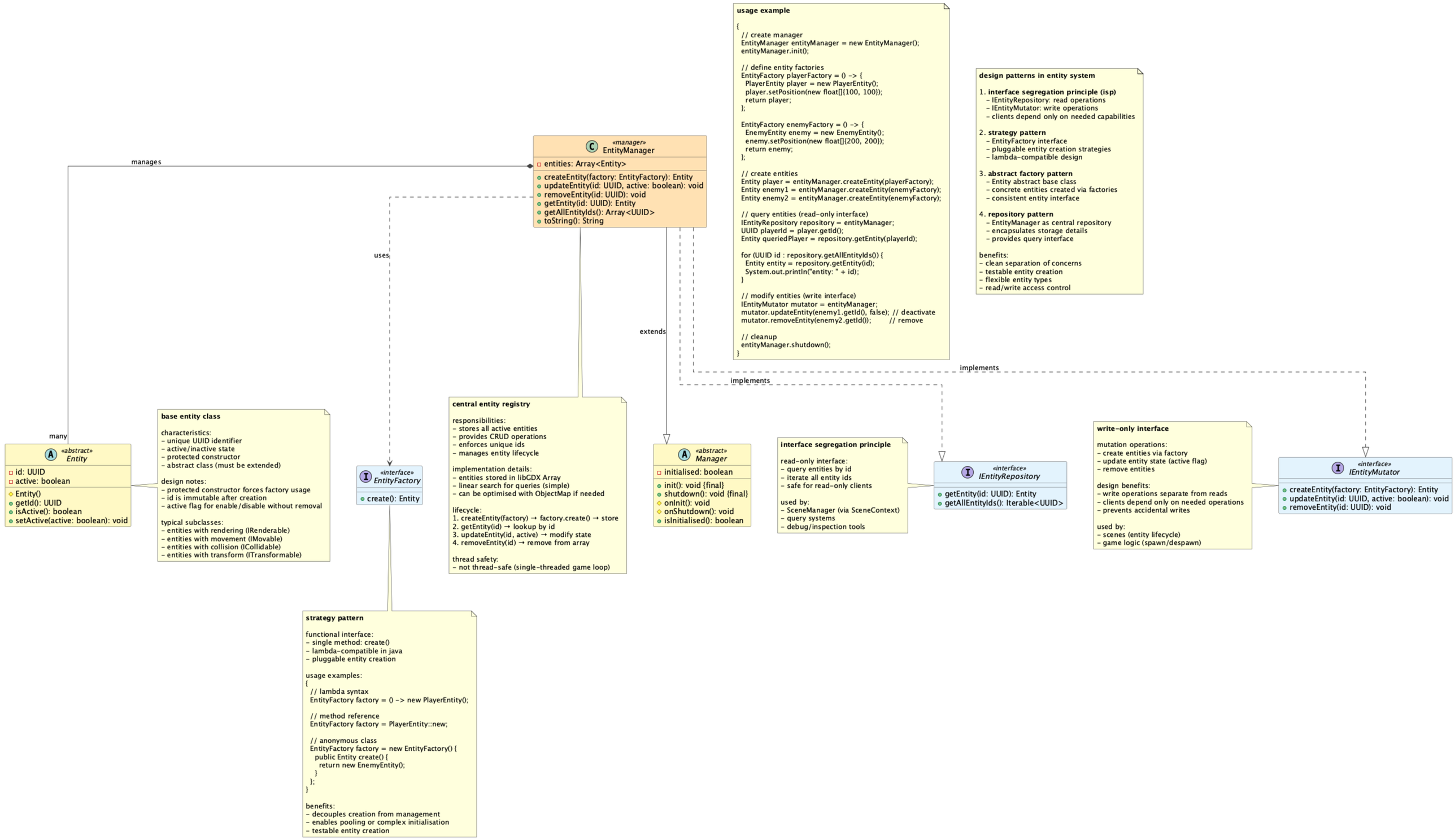
algorithm:

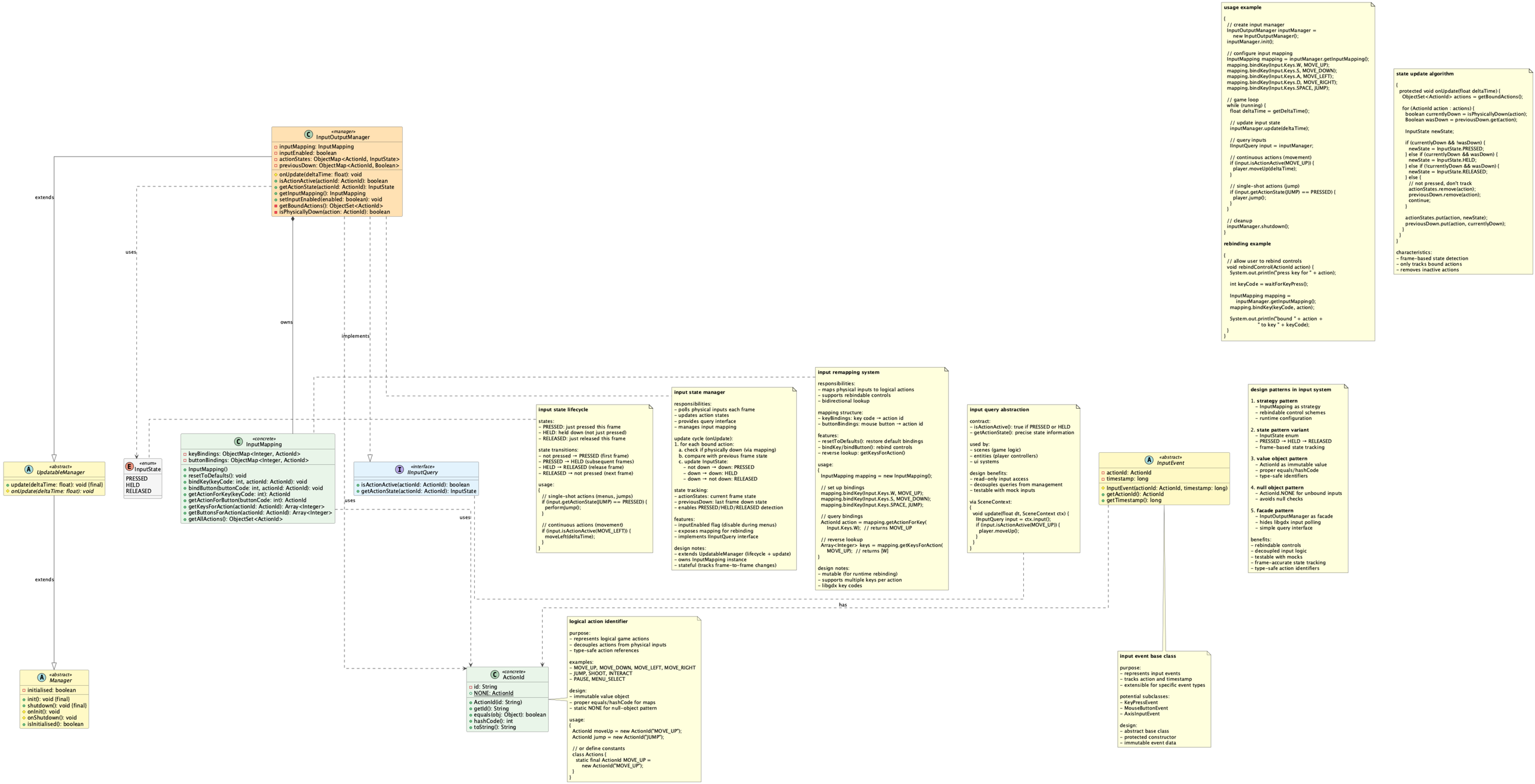
```
{
  public boolean checkCollision(
      ICollidable a, ICollidable b) {
    Rectangle boundsA = a.getBounds();
    Rectangle boundsB = b.getBounds();
    return boundsA.overlaps(boundsB);
  }
}
```

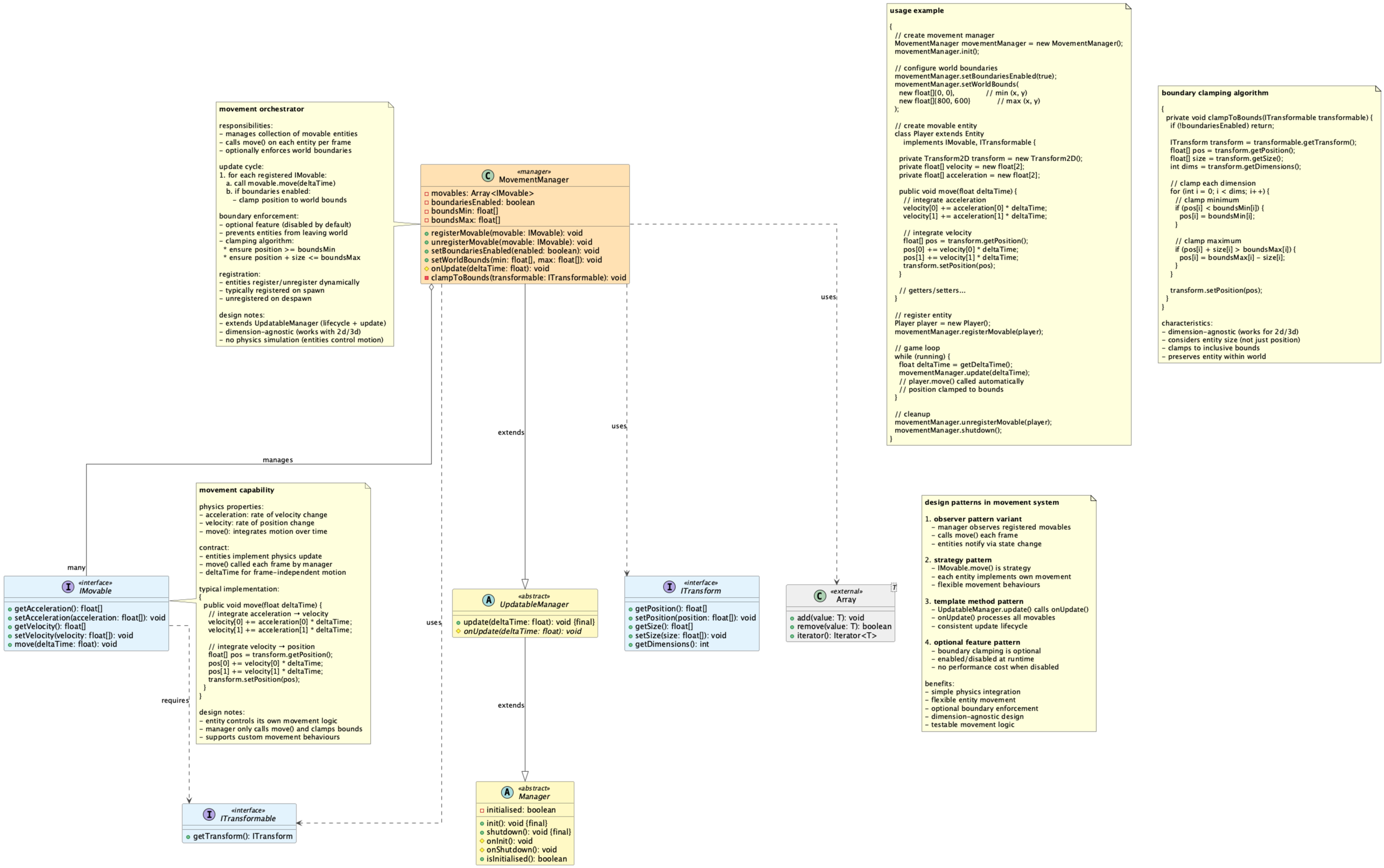
characteristics:
- simple aabb collision
- no spatial partitioning (brute force)
- sufficient for small entity counts

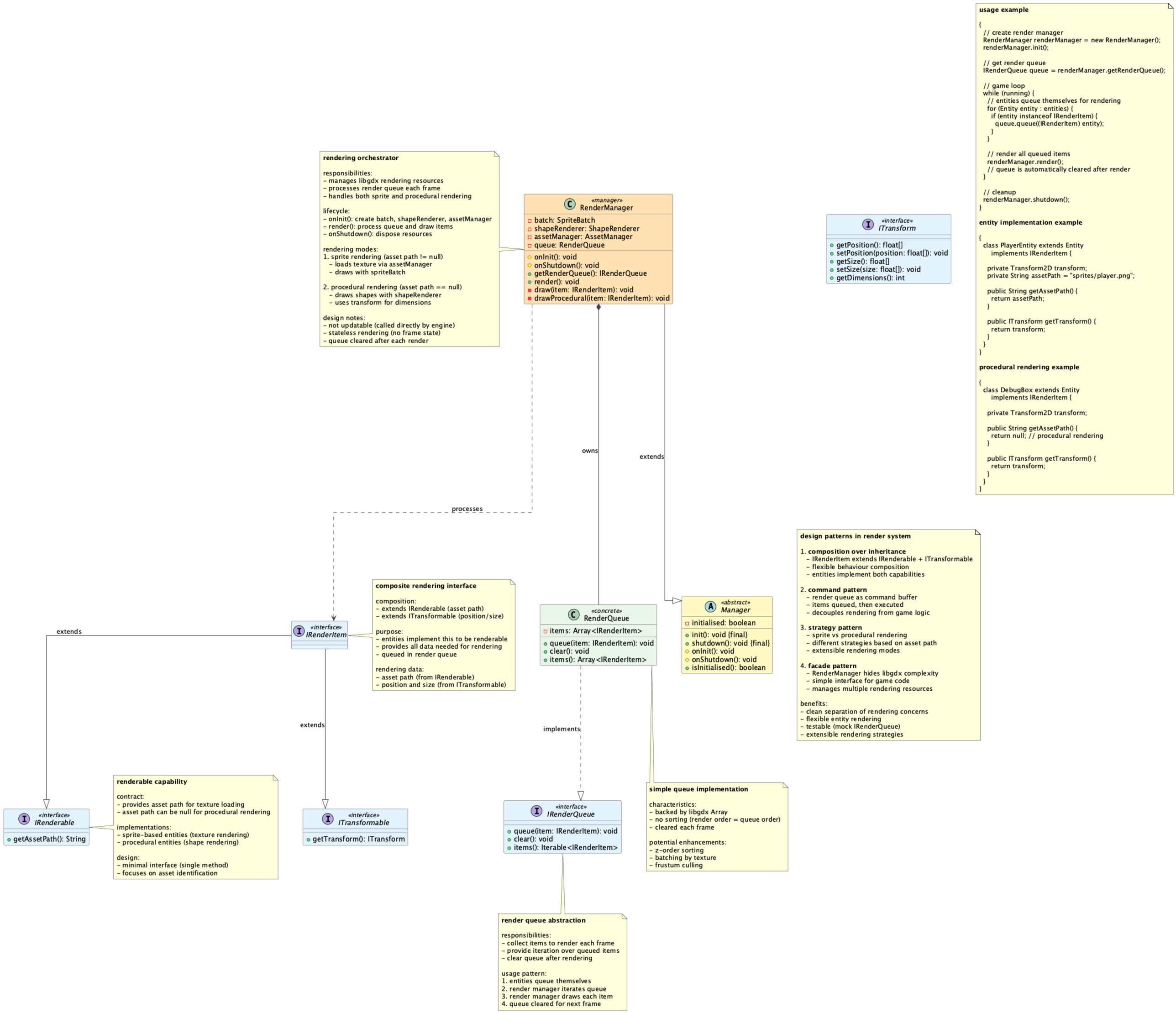
potential enhancements:
- quadtree spatial partitioning
- swept collision detection
- custom collision shapes

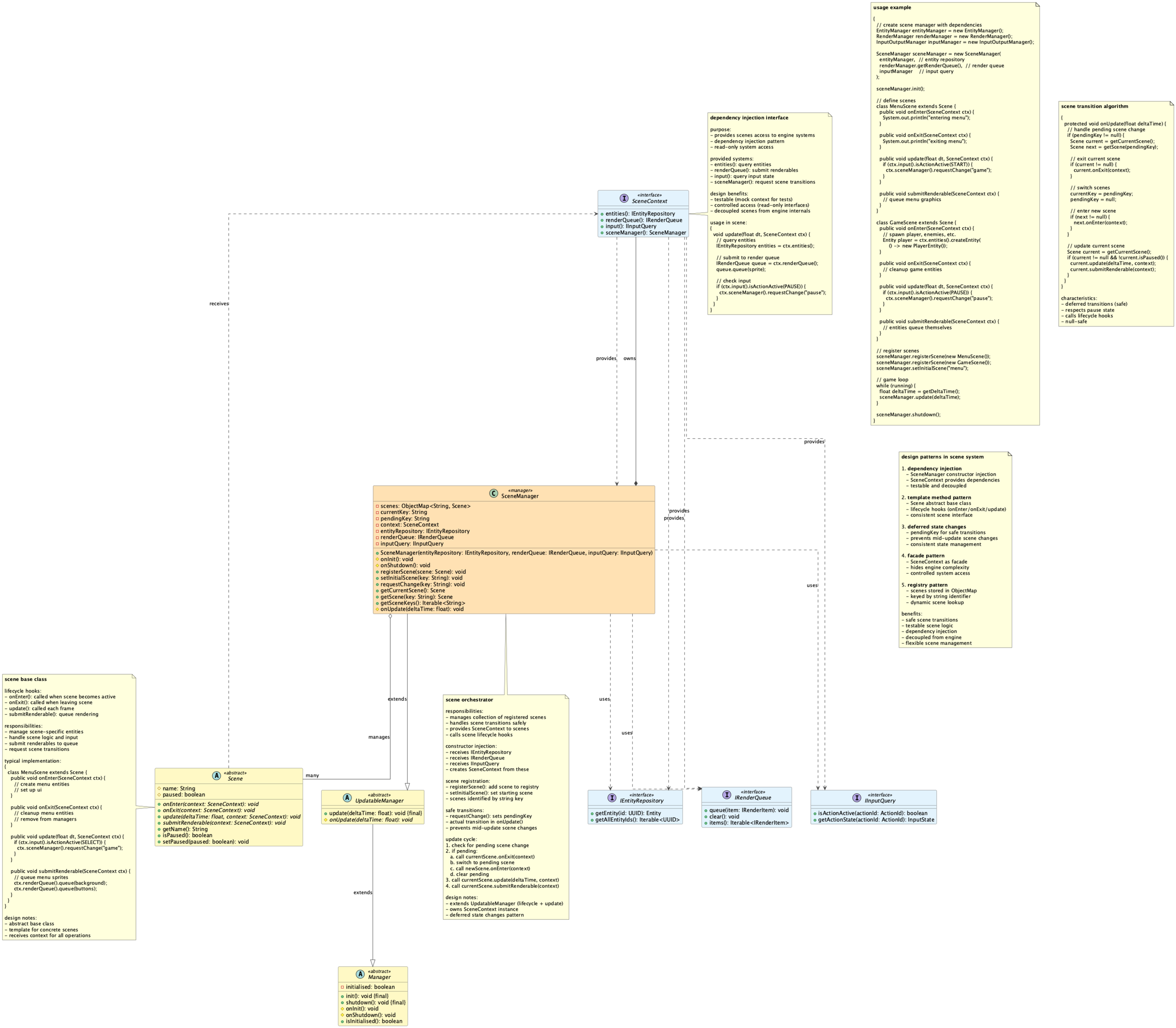

«manager» MovementManager
...
«manager» CollisionManager
...
«manager» InputOutputManager
...
«manager» SceneManager
...
«concrete» Settings
WINDOW_WIDTH: int = 640
WINDOW_HEIGHT: int = 480
usage example
{
// create engine
Engine engine = new Engine();
// create and register managers
EntityManager entityManager = new EntityManager();
RenderManager renderManager = new RenderManager();
MovementManager movementManager = new MovementManager();
CollisionManager collisionManager = new CollisionManager();
InputOutputManager inputManager = new InputOutputManager();
SceneManager sceneManager = new SceneManager(
entityManager,
renderManager.getRenderQueue(),
inputManager
);
// register with engine
engine.registerManager(entityManager);
engine.registerManager(renderManager);
engine.registerManager(movementManager);
engine.registerManager(collisionManager);
engine.registerManager(inputManager);
engine.registerManager(sceneManager);
// register updatables
engine.registerUpdatable(movementManager);
engine.registerUpdatable(collisionManager);
engine.registerUpdatable(inputManager);
engine.registerUpdatable(sceneManager);
// set render manager
engine.setRenderManager(renderManager);
// initialise all subsystems
engine.init();
// game loop
while (running) {
float deltaTime = getDeltaTime();
engine.update(deltaTime);
engine.render();
}
// cleanup
engine.shutdown();
}
«manager» EntityManager
...
manager implementations
non-updatable managers:
- EntityManager (manages entities)
- RenderManager (rendering pipeline)
updatable managers:
- MovementManager (physics updates)
- CollisionManager (collision detection)
- InputOutputManager (input polling)
- SceneManager (scene transitions)
see individual package diagrams
for detailed implementations
«manager» RenderManager
...
«abstract» UpdatableManager
update(deltaTime: float): void {final}
onUpdate(deltaTime: float): void
central orchestrator
responsibilities:
- manages all subsystem managers
- coordinates lifecycle (init → update → shutdown)
- delegates update to updatable managers
- delegates rendering to RenderManager
lifecycle flow:
1. register all managers
2. init() - initialise all managers
3. game loop:
- update(deltaTime) - update all updatables
- render() - render via RenderManager
4. shutdown() - cleanup all managers
design notes:
- composition over inheritance
- managers registered dynamically
- render manager separate (not updatable)
«concrete» Engine
managers: Array<IManager>
updatables: Array<IUpdatable>
renderManager: RenderManager
registerManager(manager: IManager): void
registerUpdatable(updatable: IUpdatable): void
setRenderManager(renderManager: RenderManager): void
init(): void
update(deltaTime: float): void
render(): void
shutdown(): void
global configuration
- static constants for window size
- used by rendering system
- can be extended for other settings
updates
many
«abstract» Manager
initialised: boolean
init(): void {final}
shutdown(): void {final}
onInit(): void
onShutdown(): void
isInitialised(): boolean
updatable extension
update pattern:
- extends Manager with update capability
- update() → onUpdate(deltaTime)
- onUpdate() is abstract
registered managers:
- MovementManager
- CollisionManager
- InputOutputManager
- SceneManager
«interface» IUpdatable
update(deltaTime: float): void
update contract
- frame-based update loop
- deltaTime for frame-independent logic
- registered with engine
template method pattern
lifecycle hooks:
- init() → onInit() (protected hook)
- shutdown() → onShutdown() (protected hook)
benefits:
- consistent lifecycle management
- subclasses override protected hooks
- final methods prevent misuse
- tracks initialisation state
usage:
1. subclass Manager
2. override onInit() and onShutdown()
3. engine calls init() before use
4. engine calls shutdown() on cleanup
manages
many
«interface» IManager
init(): void
shutdown(): void
lifecycle contract
- all managers implement init/shutdown
- ensures proper resource management
- engine orchestrates lifecycle

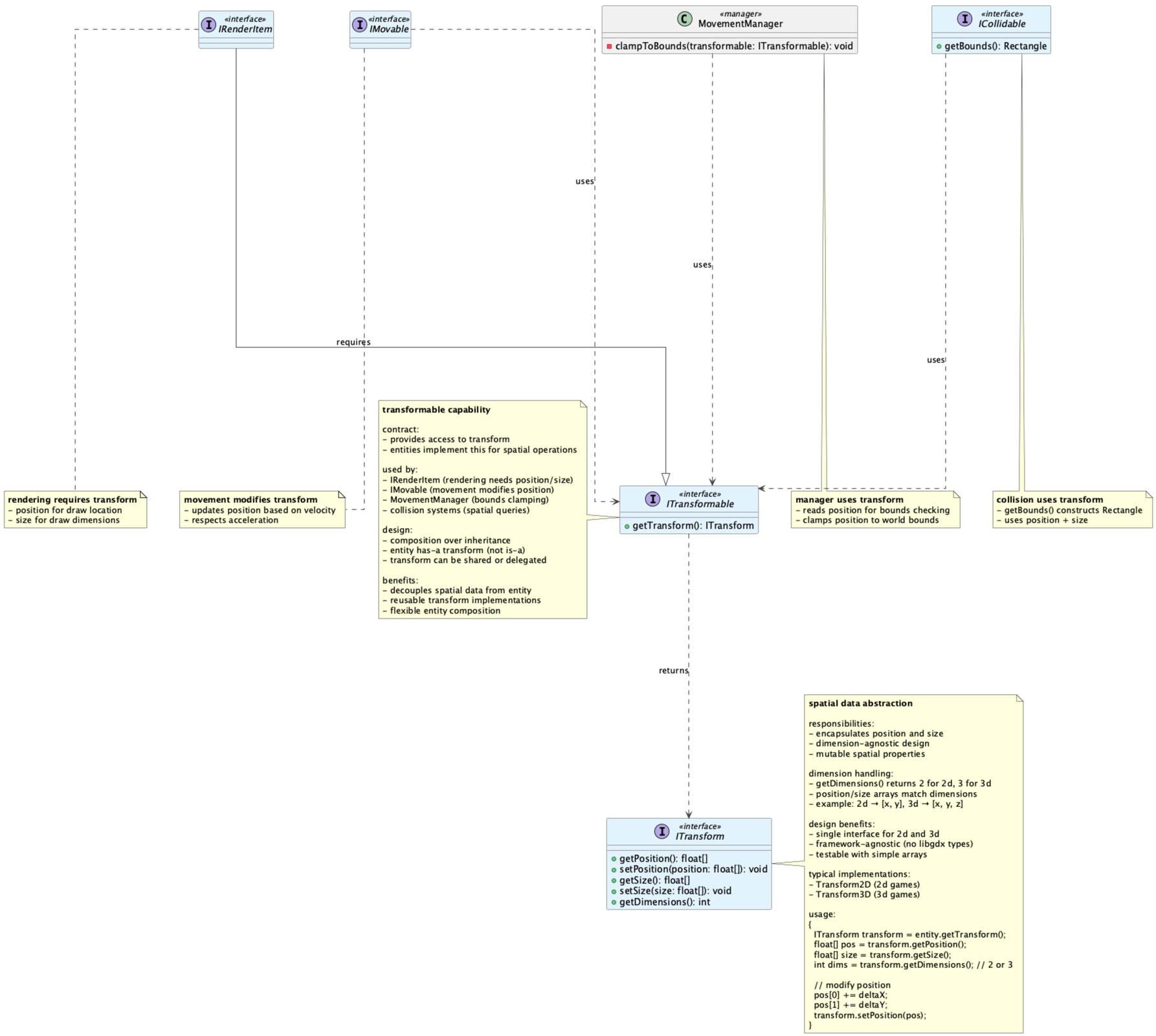

«manager»
EntityManager
entities: Array<Entity>
createEntity(factory: EntityFactory): Entity
updateEntity(id: UUID, active: boolean): void
removeEntity(id: UUID): void
getEntity(id: UUID): Entity
getAllEntityIds(): Array<UUID>
toString(): String
manages
many
uses
extends
implements
implements
«abstract»
Entity
id: UUID
active: boolean
Entity()
getId(): UUID
isActive(): boolean
setActive(active: boolean): void
«interface»
EntityFactory
create(): Entity
«abstract»
Manager
initialised: boolean
init(): void {final}
shutdown(): void {final}
onInit(): void
onShutdown(): void
isInitialised(): boolean
«interface»
IEntityRepository
getEntity(id: UUID): Entity
getAllEntityIds(): Iterable<UUID>
«interface»
IEntityMutator
createEntity(factory: EntityFactory): Entity
updateEntity(id: UUID, active: boolean): void
removeEntity(id: UUID): void
usage example
{
// create manager
EntityManager entityManager = new EntityManager();
entityManager.init();
// define entity factories
EntityFactory playerFactory = () -> {
PlayerEntity player = new PlayerEntity();
player.setPosition(new float[]{100, 100});
return player;
};
EntityFactory enemyFactory = () -> {
EnemyEntity enemy = new EnemyEntity();
enemy.setPosition(new float[]{200, 200});
return enemy;
};
// create entities
Entity player = entityManager.createEntity(playerFactory);
Entity enemy1 = entityManager.createEntity(enemyFactory);
Entity enemy2 = entityManager.createEntity(enemyFactory);
// query entities (read-only interface)
IEntityRepository repository = entityManager;
UUID playerId = player.getId();
Entity queriedPlayer = repository.getEntity(playerId);
for (UUID id : repository.getAllEntityIds()) {
Entity entity = repository.getEntity(id);
System.out.println("entity: " + id);
}
// modify entities (write interface)
IEntityMutator mutator = entityManager;
mutator.updateEntity(enemy1.getId(), false); // deactivate
mutator.removeEntity(enemy2.getId()); // remove
// cleanup
entityManager.shutdown();
}
design patterns in entity system
1. interface segregation principle (isp)
- IEntityRepository: read operations
- IEntityMutator: write operations
- clients depend only on needed capabilities
2. strategy pattern
- EntityFactory interface
- pluggable entity creation strategies
- lambda-compatible design
3. abstract factory pattern
- Entity abstract base class
- concrete entities created via factories
- consistent entity interface
4. repository pattern
- EntityManager as central repository
- encapsulates storage details
- provides query interface
benefits:
- clean separation of concerns
- testable entity creation
- flexible entity types
- read/write access control
base entity class
characteristics:
- unique UUID identifier
- active/inactive state
- protected constructor
- abstract class (must be extended)
design notes:
- protected constructor forces factory usage
- id is immutable after creation
- active flag for enable/disable without removal
typical subclasses:
- entities with rendering (IRenderable)
- entities with movement (IMovable)
- entities with collision (ICollidable)
- entities with transform (ITransformable)
central entity registry
responsibilities:
- stores all active entities
- provides CRUD operations
- enforces unique ids
- manages entity lifecycle
implementation details:
- entities stored in libGDX Array
- linear search for queries (simple)
- can be optimised with ObjectMap if needed
lifecycle:
1. createEntity(factory) → factory.create() → store
2. getEntity(id) → lookup by id
3. updateEntity(id, active) → modify state
4. removeEntity(id) → remove from array
thread safety:
- not thread-safe (single-threaded game loop)
interface segregation principle
read-only interface:
- query entities by id
- iterate all entity ids
- safe for read-only clients
used by:
- SceneManager (via SceneContext)
- query systems
- debug/inspection tools
write-only interface
mutation operations:
- create entities via factory
- update entity state (active flag)
- remove entities
design benefits:
- write operations separate from reads
- clients depend only on needed operations
- prevents accidental writes
used by:
- scenes (entity lifecycle)
- game logic (spawn/despawn)
strategy pattern
functional interface:
- single method: create()
- lambda-compatible in java
- pluggable entity creation
usage examples:
{
// lambda syntax
EntityFactory factory = () -> new PlayerEntity();
// method reference
EntityFactory factory = PlayerEntity::new;
// anonymous class
EntityFactory factory = new EntityFactory() {
public Entity create() {
return new EnemyEntity();
}
};
}
benefits:
- decouples creation from management
- enables pooling or complex initialisation
- testable entity creation


«manager»
InputOutputManager
inputMapping: InputMapping
inputEnabled: boolean
actionStates: ObjectMap<ActionId, InputState>
previousDown: ObjectMap<ActionId, Boolean>
onUpdate(deltaTime: float): void
isActionActive(actionId: ActionId): boolean
getActionState(actionId: ActionId): InputState
getInputMapping(): InputMapping
setInputEnabled(enabled: boolean): void
getBoundActions(): ObjectSet<ActionId>
isPhysicallyDown(action: ActionId): boolean
extends
uses
owns
implements
«abstract»
UpdatableManager
update(deltaTime: float): void {final}
onUpdate(deltaTime: float): void
«enum»
InputState
PRESSED
HELD
RELEASED
«concrete»
InputMapping
keyBindings: ObjectMap<Integer, ActionId>
buttonBindings: ObjectMap<Integer, ActionId>
InputMapping()
resetToDefaults(): void
bindKey(keyCode: int, actionId: ActionId): void
bindButton(buttonCode: int, actionId: ActionId): void
getActionForKey(keyCode: int): ActionId
getActionForButton(buttonCode: int): ActionId
getKeysForAction(actionId: ActionId): Array<Integer>
getButtonsForAction(actionId: ActionId): Array<Integer>
getAllActions(): ObjectSet<ActionId>
«interface»
IInputQuery
isActionActive(actionId: ActionId): boolean
getActionState(actionId: ActionId): InputState
uses
uses
extends
«abstract»
Manager
initialised: boolean
init(): void {final}
shutdown(): void {final}
onInit(): void
onShutdown(): void
isInitialised(): boolean
«concrete»
ActionId
id: String
NONE: ActionId
ActionId(id: String)
getId(): String
equals(obj: Object): boolean
hashCode(): int
toString(): String
has
«abstract»
InputEvent
actionId: ActionId
timestamp: long
InputEvent(actionId: ActionId, timestamp: long)
getActionId(): ActionId
getTimestamp(): long
input event base class
purpose:
- represents input events
- tracks action and timestamp
- extensible for specific event types
potential subclasses:
- KeyPressEvent
- MouseButtonEvent
- AxisInputEvent
design:
- abstract base class
- protected constructor
- immutable event data
logical action identifier
purpose:
- represents logical game actions
- decouples actions from physical inputs
- type-safe action references
examples:
- MOVE_UP, MOVE_DOWN, MOVE_LEFT, MOVE_RIGHT
- JUMP, SHOOT, INTERACT
- PAUSE, MENU_SELECT
design:
- immutable value object
- proper equals/hashCode for maps
- static NONE for null-object pattern
usage:
{
ActionId moveUp = new ActionId("MOVE_UP");
ActionId jump = new ActionId("JUMP");
// or define constants
class Actions {
static final ActionId MOVE_UP =
new ActionId("MOVE_UP");
}
}
input state lifecycle
states:
- PRESSED: just pressed this frame
- HELD: held down (not just pressed)
- RELEASED: just released this frame
state transitions:
- not pressed → PRESSED (first frame)
- PRESSED → HELD (subsequent frames)
- HELD → RELEASED (release frame)
- RELEASED → not pressed (next frame)
usage:
{
// single-shot actions (menus, jumps)
if (input.getActionState(JUMP) == PRESSED) {
performJump();
}
// continuous actions (movement)
if (input.isActionActive(MOVE_LEFT)) {
moveLeft(deltaTime);
}
}
input state manager
responsibilities:
- polls physical inputs each frame
- updates action states
- provides query interface
- manages input mapping
update cycle (onUpdate):
1. for each bound action:
a. check if physically down (via mapping)
b. compare with previous frame state
c. update InputState:
- not down → down: PRESSED
- down → down: HELD
- down → not down: RELEASED
state tracking:
- actionStates: current frame state
- previousDown: last frame down state
- enables PRESSED/HELD/RELEASED detection
features:
- inputEnabled flag (disable during menus)
- exposes mapping for rebinding
- implements IInputQuery interface
design notes:
- extends UpdatableManager (lifecycle + update)
- owns InputMapping instance
- stateful (tracks frame-to-frame changes)
input remapping system
responsibilities:
- maps physical inputs to logical actions
- supports rebindable controls
- bidirectional lookup
mapping structure:
- keyBindings: key code → action id
- buttonBindings: mouse button → action id
features:
- resetToDefaults(): restore default bindings
- bindKey/bindButton(): rebind controls
- reverse lookup: getKeysForAction()
usage:
{
InputMapping mapping = new InputMapping();
// set up bindings
mapping.bindKey(Input.Keys.W, MOVE_UP);
mapping.bindKey(Input.Keys.S, MOVE_DOWN);
mapping.bindKey(Input.Keys.SPACE, JUMP);
// query bindings
ActionId action = mapping.getActionForKey(
Input.Keys.W); // returns MOVE_UP
// reverse lookup
Array<Integer> keys = mapping.getKeysForAction(
MOVE_UP); // returns [W]
}
design notes:
- mutable (for runtime rebinding)
- supports multiple keys per action
- libgdx key codes
input query abstraction
contract:
- isActionActive(): true if PRESSED or HELD
- getActionState(): precise state information
used by:
- scenes (game logic)
- entities (player controllers)
- ui systems
design benefits:
- read-only input access
- decouples queries from management
- testable with mock inputs
via SceneContext:
{
void update(float dt, SceneContext ctx) {
IInputQuery input = ctx.input();
if (input.isActionActive(MOVE_UP)) {
player.moveUp();
}
}
}
design patterns in input system
1. strategy pattern
- InputMapping as strategy
- rebindable control schemes
- runtime configuration
2. state pattern variant
- InputState enum
- PRESSED → HELD → RELEASED
- frame-based state tracking
3. value object pattern
- ActionId as immutable value
- proper equals/hashCode
- type-safe identifiers
4. null object pattern
- ActionId.NONE for unbound inputs
- avoids null checks
5. facade pattern
- InputOutputManager as facade
- hides libgdx input polling
- simple query interface
benefits:
- rebindable controls
- decoupled input logic
- testable with mocks
- frame-accurate state tracking
- type-safe action identifiers
usage example
{
// create input manager
InputOutputManager inputManager =
new InputOutputManager();
inputManager.init();
// configure input mapping
InputMapping mapping = inputManager.getInputMapping();
mapping.bindKey(Input.Keys.W, MOVE_UP);
mapping.bindKey(Input.Keys.S, MOVE_DOWN);
mapping.bindKey(Input.Keys.A, MOVE_LEFT);
mapping.bindKey(Input.Keys.D, MOVE_RIGHT);
mapping.bindKey(Input.Keys.SPACE, JUMP);
// game loop
while (running) {
float deltaTime = getDeltaTime();
// update input state
inputManager.update(deltaTime);
// query inputs
IInputQuery input = inputManager;
// continuous actions (movement)
if (input.isActionActive(MOVE_UP)) {
player.moveUp(deltaTime);
}
// single-shot actions (jump)
if (input.getActionState(JUMP) == PRESSED) {
player.jump();
}
}
// cleanup
inputManager.shutdown();
}
rebinding example
{
// allow user to rebind controls
void rebindControl(ActionId action) {
System.out.println("press key for " + action);
int keyCode = waitForKeyPress();
InputMapping mapping =
inputManager.getInputMapping();
mapping.bindKey(keyCode, action);
System.out.println("bound " + action +
" to key " + keyCode);
}
}
state update algorithm
{
protected void onUpdate(float deltaTime) {
ObjectSet<ActionId> actions = getBoundActions();
for (ActionId action : actions) {
boolean currentlyDown = isPhysicallyDown(action);
Boolean wasDown = previousDown.get(action);
InputState newState;
if (currentlyDown && !wasDown) {
newState = InputState.PRESSED;
} else if (currentlyDown && wasDown) {
newState = InputState.HELD;
} else if (!currentlyDown && wasDown) {
newState = InputState.RELEASED;
} else {
// not pressed, don't track
actionStates.remove(action);
previousDown.remove(action);
continue;
}
actionStates.put(action, newState);
previousDown.put(action, currentlyDown);
}
}
}
characteristics:
- frame-based state detection
- only tracks bound actions
- removes inactive actions


movement orchestrator
responsibilities:
- manages collection of movable entities
- calls move() on each entity per frame
- optionally enforces world boundaries
update cycle:
1. for each registered IMovable:
a. call movable.move(deltaTime)
b. if boundaries enabled:
- clamp position to world bounds
boundary enforcement:
- optional feature (disabled by default)
- prevents entities from leaving world
- clamping algorithm:
* ensure position >= boundsMin
* ensure position + size <= boundsMax
registration:
- entities register/unregister dynamically
- typically registered on spawn
- unregistered on despawn
design notes:
- extends UpdatableManager (lifecycle + update)
- dimension-agnostic (works with 2d/3d)
- no physics simulation (entities control motion)
«manager»
MovementManager
movables: Array<IMovable>
boundariesEnabled: boolean
boundsMin: float[]
boundsMax: float[]
registerMovable(movable: IMovable): void
unregisterMovable(movable: IMovable): void
setBoundariesEnabled(enabled: boolean): void
setWorldBounds(min: float[], max: float[]): void
onUpdate(deltaTime: float): void
clampToBounds(transformable: ITransformable): void
usage example
{
// create movement manager
MovementManager movementManager = new MovementManager();
movementManager.init();
// configure world boundaries
movementManager.setBoundariesEnabled(true);
movementManager.setWorldBounds(
new float[]{0, 0}, // min (x, y)
new float[]{800, 600} // max (x, y)
);
// create movable entity
class Player extends Entity
implements IMovable, ITransformable {
private Transform2D transform = new Transform2D();
private float[] velocity = new float[2];
private float[] acceleration = new float[2];
public void move(float deltaTime) {
// integrate acceleration
velocity[0] += acceleration[0] * deltaTime;
velocity[1] += acceleration[1] * deltaTime;
// integrate velocity
float[] pos = transform.getPosition();
pos[0] += velocity[0] * deltaTime;
pos[1] += velocity[1] * deltaTime;
transform.setPosition(pos);
}
// getters/setters...
}
// register entity
Player player = new Player();
movementManager.registerMovable(player);
// game loop
while (running) {
float deltaTime = getDeltaTime();
movementManager.update(deltaTime);
// player.move() called automatically
// position clamped to bounds
}
// cleanup
movementManager.unregisterMovable(player);
movementManager.shutdown();
}
boundary clamping algorithm
{
private void clampToBounds(ITransformable transformable) {
if (!boundariesEnabled) return;
ITransform transform = transformable.getTransform();
float[] pos = transform.getPosition();
float[] size = transform.getSize();
int dims = transform.getDimensions();
// clamp each dimension
for (int i = 0; i < dims; i++) {
// clamp minimum
if (pos[i] < boundsMin[i]) {
pos[i] = boundsMin[i];
}
// clamp maximum
if (pos[i] + size[i] > boundsMax[i]) {
pos[i] = boundsMax[i] - size[i];
}
}
transform.setPosition(pos);
}
}
characteristics:
- dimension-agnostic (works for 2d/3d)
- considers entity size (not just position)
- clamps to inclusive bounds
- preserves entity within world
uses
extends
uses
manages
movement capability
physics properties:
- acceleration: rate of velocity change
- velocity: rate of position change
- move(): integrates motion over time
contract:
- entities implement physics update
- move() called each frame by manager
- deltaTime for frame-independent motion
typical implementation:
{
public void move(float deltaTime) {
// integrate acceleration → velocity
velocity[0] += acceleration[0] * deltaTime;
velocity[1] += acceleration[1] * deltaTime;
// integrate velocity → position
float[] pos = transform.getPosition();
pos[0] += velocity[0] * deltaTime;
pos[1] += velocity[1] * deltaTime;
transform.setPosition(pos);
}
}
design notes:
- entity controls its own movement logic
- manager only calls move() and clamps bounds
- supports custom movement behaviours
many
«interface»
IMovable
getAcceleration(): float[]
setAcceleration(acceleration: float[]): void
getVelocity(): float[]
setVelocity(velocity: float[]): void
move(deltaTime: float): void
uses
«abstract»
UpdatableManager
update(deltaTime: float): void {final}
onUpdate(deltaTime: float): void
«interface»
ITransform
getPosition(): float[]
setPosition(position: float[]): void
getSize(): float[]
setSize(size: float[]): void
getDimensions(): int
«external»
Array
T
add(value: T): void
remove(value: T): boolean
iterator(): Iterator<T>
design patterns in movement system
1. observer pattern variant
- manager observes registered movables
- calls move() each frame
- entities notify via state change
2. strategy pattern
- IMovable.move() is strategy
- each entity implements own movement
- flexible movement behaviours
3. template method pattern
- UpdatableManager.update() calls onUpdate()
- onUpdate() processes all movables
- consistent update lifecycle
4. optional feature pattern
- boundary clamping is optional
- enabled/disabled at runtime
- no performance cost when disabled
benefits:
- simple physics integration
- flexible entity movement
- optional boundary enforcement
- dimension-agnostic design
- testable movement logic
requires
«interface»
ITransformable
getTransform(): ITransform
extends
«abstract»
Manager
initialised: boolean
init(): void {final}
shutdown(): void {final}
onInit(): void
onShutdown(): void
isInitialised(): boolean


rendering orchestrator
responsibilities:
- manages libgdx rendering resources
- processes render queue each frame
- handles both sprite and procedural rendering
lifecycle:
- onInit(): create batch, shapeRenderer, assetManager
- render(): process queue and draw items
- onShutdown(): dispose resources
rendering modes:
1. sprite rendering (asset path != null)
- loads texture via assetManager
- draws with spriteBatch
2. procedural rendering (asset path == null)
- draws shapes with shapeRenderer
- uses transform for dimensions
design notes:
- not updatable (called directly by engine)
- stateless rendering (no frame state)
- queue cleared after each render
«manager»
RenderManager
batch: SpriteBatch
shapeRenderer: ShapeRenderer
assetManager: AssetManager
queue: RenderQueue
onInit(): void
onShutdown(): void
getRenderQueue(): IRenderQueue
render(): void
draw(item: IRenderItem): void
drawProcedural(item: IRenderItem): void
«interface»
ITransform
getPosition(): float[]
setPosition(position: float[]): void
getSize(): float[]
setSize(size: float[]): void
getDimensions(): int
usage example
{
// create render manager
RenderManager renderManager = new RenderManager();
renderManager.init();
// get render queue
IRenderQueue queue = renderManager.getRenderQueue();
// game loop
while (running) {
// entities queue themselves for rendering
for (Entity entity : entities) {
if (entity instanceof IRenderItem) {
queue.queue((IRenderItem) entity);
}
}
// render all queued items
renderManager.render();
// queue is automatically cleared after render
}
// cleanup
renderManager.shutdown();
}
entity implementation example
{
class PlayerEntity extends Entity
implements IRenderItem {
private Transform2D transform;
private String assetPath = "sprites/player.png";
public String getAssetPath() {
return assetPath;
}
public ITransform getTransform() {
return transform;
}
}
}
procedural rendering example
{
class DebugBox extends Entity
implements IRenderItem {
private Transform2D transform;
public String getAssetPath() {
return null; // procedural rendering
}
public ITransform getTransform() {
return transform;
}
}
}
owns
extends
processes
«interface»
IRenderItem
composite rendering interface
composition:
- extends IRenderable (asset path)
- extends ITransformable (position/size)
purpose:
- entities implement this to be renderable
- provides all data needed for rendering
- queued in render queue
rendering data:
- asset path (from IRenderable)
- position and size (from ITransformable)
«concrete»
RenderQueue
items: Array<IRenderItem>
queue(item: IRenderItem): void
clear(): void
items(): Array<IRenderItem>
«abstract»
Manager
initialised: boolean
init(): void {final}
shutdown(): void {final}
onInit(): void
onShutdown(): void
isInitialised(): boolean
design patterns in render system
1. composition over inheritance
- IRenderItem extends IRenderable + ITransformable
- flexible behaviour composition
- entities implement both capabilities
2. command pattern
- render queue as command buffer
- items queued, then executed
- decouples rendering from game logic
3. strategy pattern
- sprite vs procedural rendering
- different strategies based on asset path
- extensible rendering modes
4. facade pattern
- RenderManager hides libgdx complexity
- simple interface for game code
- manages multiple rendering resources
benefits:
- clean separation of rendering concerns
- flexible entity rendering
- testable (mock IRenderQueue)
- extensible rendering strategies
extends
extends
implements
«interface»
IRenderable
getAssetPath(): String
renderable capability
contract:
- provides asset path for texture loading
- asset path can be null for procedural rendering
implementations:
- sprite-based entities (texture rendering)
- procedural entities (shape rendering)
design:
- minimal interface (single method)
- focuses on asset identification
«interface»
ITransformable
getTransform(): ITransform
«interface»
IRenderQueue
queue(item: IRenderItem): void
clear(): void
items(): Iterable<IRenderItem>
simple queue implementation
characteristics:
- backed by libgdx Array
- no sorting (render order = queue order)
- cleared each frame
potential enhancements:
- z-order sorting
- batching by texture
- frustum culling
render queue abstraction
responsibilities:
- collect items to render each frame
- provide iteration over queued items
- clear queue after rendering
usage pattern:
1. entities queue themselves
2. render manager iterates queue
3. render manager draws each item
4. queue cleared for next frame


usage example
{
// create scene manager with dependencies
EntityManager entityManager = new EntityManager();
RenderManager renderManager = new RenderManager();
InputOutputManager inputManager = new InputOutputManager();
SceneManager sceneManager = new SceneManager(
entityManager, // entity repository
renderManager.getRenderQueue(), // render queue
inputManager // input query
);
sceneManager.init();
// define scenes
class MenuScene extends Scene {
public void onEnter(SceneContext ctx) {
System.out.println("entering menu");
}
public void onExit(SceneContext ctx) {
System.out.println("exiting menu");
}
public void update(float dt, SceneContext ctx) {
if (ctx.input().isActionActive(START)) {
ctx.sceneManager().requestChange("game");
}
}
public void submitRenderable(SceneContext ctx) {
// queue menu graphics
}
}
class GameScene extends Scene {
public void onEnter(SceneContext ctx) {
// spawn player, enemies, etc.
Entity player = ctx.entities().createEntity(
() -> new PlayerEntity());
}
public void onExit(SceneContext ctx) {
// cleanup game entities
}
public void update(float dt, SceneContext ctx) {
if (ctx.input().isActionActive(PAUSE)) {
ctx.sceneManager().requestChange("pause");
}
}
public void submitRenderable(SceneContext ctx) {
// entities queue themselves
}
}
// register scenes
sceneManager.registerScene(new MenuScene());
sceneManager.registerScene(new GameScene());
sceneManager.setInitialScene("menu");
// game loop
while (running) {
float deltaTime = getDeltaTime();
sceneManager.update(deltaTime);
}
sceneManager.shutdown();
}
scene transition algorithm
{
protected void onUpdate(float deltaTime) {
// handle pending scene change
if (pendingKey != null) {
Scene current = getCurrentScene();
Scene next = getScene(pendingKey);
// exit current scene
if (current != null) {
current.onExit(context);
}
// switch scenes
currentKey = pendingKey;
pendingKey = null;
// enter new scene
if (next != null) {
next.onEnter(context);
}
}
// update current scene
Scene current = getCurrentScene();
if (current != null && !current.isPaused()) {
current.update(deltaTime, context);
current.submitRenderable(context);
}
}
}
characteristics:
- deferred transitions (safe)
- respects pause state
- calls lifecycle hooks
- null-safe
dependency injection interface
purpose:
- provides scenes access to engine systems
- dependency injection pattern
- read-only system access
provided systems:
- entities(): query entities
- renderQueue(): submit renderables
- input(): query input state
- sceneManager(): request scene transitions
design benefits:
- testable (mock context for tests)
- controlled access (read-only interfaces)
- decoupled scenes from engine internals
usage in scene:
{
void update(float dt, SceneContext ctx) {
// query entities
IEntityRepository entities = ctx.entities();
// submit to render queue
IRenderQueue queue = ctx.renderQueue();
queue.queue(sprite);
// check input
if (ctx.input().isActionActive(PAUSE)) {
ctx.sceneManager().requestChange("pause");
}
}
}
«interface»
SceneContext
entities(): IEntityRepository
renderQueue(): IRenderQueue
input(): IInputQuery
sceneManager(): SceneManager
receives
provides
owns
provides
provides
provides
«manager»
SceneManager
scenes: ObjectMap<String, Scene>
currentKey: String
pendingKey: String
context: SceneContext
entityRepository: IEntityRepository
renderQueue: IRenderQueue
inputQuery: IInputQuery
SceneManager(entityRepository: IEntityRepository, renderQueue: IRenderQueue, inputQuery: IInputQuery)
onInit(): void
onShutdown(): void
registerScene(scene: Scene): void
setInitialScene(key: String): void
requestChange(key: String): void
getCurrentScene(): Scene
getScene(key: String): Scene
getSceneKeys(): Iterable<String>
onUpdate(deltaTime: float): void
uses
design patterns in scene system
1. dependency injection
- SceneManager constructor injection
- SceneContext provides dependencies
- testable and decoupled
2. template method pattern
- Scene abstract base class
- lifecycle hooks (onEnter/onExit/update)
- consistent scene interface
3. deferred state changes
- pendingKey for safe transitions
- prevents mid-update scene changes
- consistent state management
4. facade pattern
- SceneContext as facade
- hides engine complexity
- controlled system access
5. registry pattern
- scenes stored in ObjectMap
- keyed by string identifier
- dynamic scene lookup
benefits:
- safe scene transitions
- testable scene logic
- dependency injection
- decoupled from engine
- flexible scene management
scene base class
lifecycle hooks:
- onEnter(): called when scene becomes active
- onExit(): called when leaving scene
- update(): called each frame
- submitRenderable(): queue rendering
responsibilities:
- manage scene-specific entities
- handle scene logic and input
- submit renderables to queue
- request scene transitions
typical implementation:
{
class MenuScene extends Scene {
public void onEnter(SceneContext ctx) {
// create menu entities
// set up ui
}
public void onExit(SceneContext ctx) {
// cleanup menu entities
// remove from managers
}
public void update(float dt, SceneContext ctx) {
if (ctx.input().isActionActive(SELECT)) {
ctx.sceneManager().requestChange("game");
}
}
public void submitRenderable(SceneContext ctx) {
// queue menu sprites
ctx.renderQueue().queue(background);
ctx.renderQueue().queue(buttons);
}
}
}
design notes:
- abstract base class
- template for concrete scenes
- receives context for all operations
extends
manages
many
«abstract»
Scene
name: String
paused: boolean
onEnter(context: SceneContext): void
onExit(context: SceneContext): void
update(deltaTime: float, context: SceneContext): void
submitRenderable(context: SceneContext): void
getName(): String
isPaused(): boolean
setPaused(paused: boolean): void
«abstract»
UpdatableManager
update(deltaTime: float): void {final}
onUpdate(deltaTime: float): void
extends
«abstract»
Manager
initialised: boolean
init(): void {final}
shutdown(): void {final}
onInit(): void
onShutdown(): void
isInitialised(): boolean
scene orchestrator
responsibilities:
- manages collection of registered scenes
- handles scene transitions safely
- provides SceneContext to scenes
- calls scene lifecycle hooks
constructor injection:
- receives IEntityRepository
- receives IRenderQueue
- receives IInputQuery
- creates SceneContext from these
scene registration:
- registerScene(): add scene to registry
- setInitialScene(): set starting scene
- scenes identified by string key
safe transitions:
- requestChange(): sets pendingKey
- actual transition in onUpdate()
- prevents mid-update scene changes
update cycle:
1. check for pending scene change
2. if pending:
a. call currentScene.onExit(context)
b. switch to pending scene
c. call newScene.onEnter(context)
d. clear pending
3. call currentScene.update(deltaTime, context)
4. call currentScene.submitRenderable(context)
design notes:
- extends UpdatableManager (lifecycle + update)
- owns SceneContext instance
- deferred state changes pattern
uses
uses
«interface»
IEntityRepository
getEntity(id: UUID): Entity
getAllEntityIds(): Iterable<UUID>
«interface»
IRenderQueue
queue(item: IRenderItem): void
clear(): void
items(): Iterable<IRenderItem>
«interface»
IInputQuery
isActionActive(actionId: ActionId): boolean
getActionState(actionId: ActionId): InputState



system interactions

systems that use ITransform:

1. **render system**
   - IRenderItem extends ITransformable
   - position determines draw location
   - size determines draw dimensions

2. **movement system**
   - IMovable entities modify position
   - MovementManager clamps to world bounds
   - uses getTransform() for position access

3. **collision system**
   - ICollidable.getBounds() uses transform
   - position + size = bounding box
   - collision detection uses spatial data

data flow:
- entity owns transform data
- systems read/modify via interface
- no direct manipulation of internals

implementation examples

2d transform

```
{
  class Transform2D implements ITransform {
    private float[] position = new float[2];
    private float[] size = new float[2];

    public float[] getPosition() {
      return position;
    }

    public void setPosition(float[] position) {
      this.position[0] = position[0];
      this.position[1] = position[1];
    }

    public float[] getSize() {
      return size;
    }

    public void setSize(float[] size) {
      this.size[0] = size[0];
      this.size[1] = size[1];
    }

    public int getDimensions() {
      return 2;
    }
  }
}
```

3d transform

```
{
  class Transform3D implements ITransform {
    private float[] position = new float[3];
    private float[] size = new float[3];

    public int getDimensions() {
      return 3;
    }

    // ... similar implementation with 3d arrays
  }
}
```

entity with transform

```
{
  class GameEntity extends Entity
      implements ITransformable {

    private ITransform transform = new Transform2D();

    public ITransform getTransform() {
      return transform;
    }
  }
}
```

design patterns in transform system

1. **strategy pattern**
   - ITransform as strategy interface
   - different implementations for 2d/3d
   - runtime dimension selection

2. **composition over inheritance**
   - ITransformable provides access
   - entity has-a transform (not is-a)
   - flexible transform implementations

3. **dependency inversion**
   - systems depend on ITransform interface
   - not coupled to concrete implementations
   - testable with mock transforms

4. **single responsibility principle**
   - ITransform: spatial data only
   - no behaviour mixed in
   - pure data container

benefits:
- dimension-agnostic architecture
- reusable across 2d and 3d
- testable spatial logic
- clean separation of concerns